

# Dimensionality Reduction

Will J. Pearson (NCBJ)  
VISTULA - 24<sup>th</sup> September 2026



Narodowe Centrum Badań Jądrowych  
National Centre for Nuclear Research



# Dimensionality Reduction

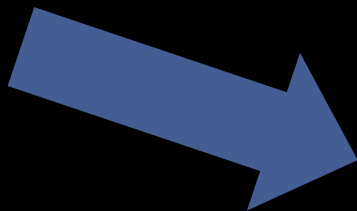
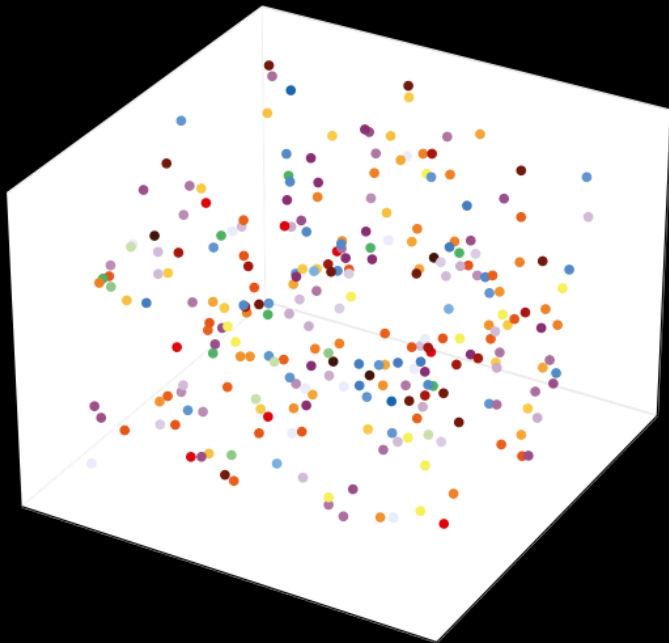
- Properties of astronomical data are often correlated
  - Sérsic Index and Concentration
  - Concentration and Gini
  - Hubble type and colour
  - Colour and SFR

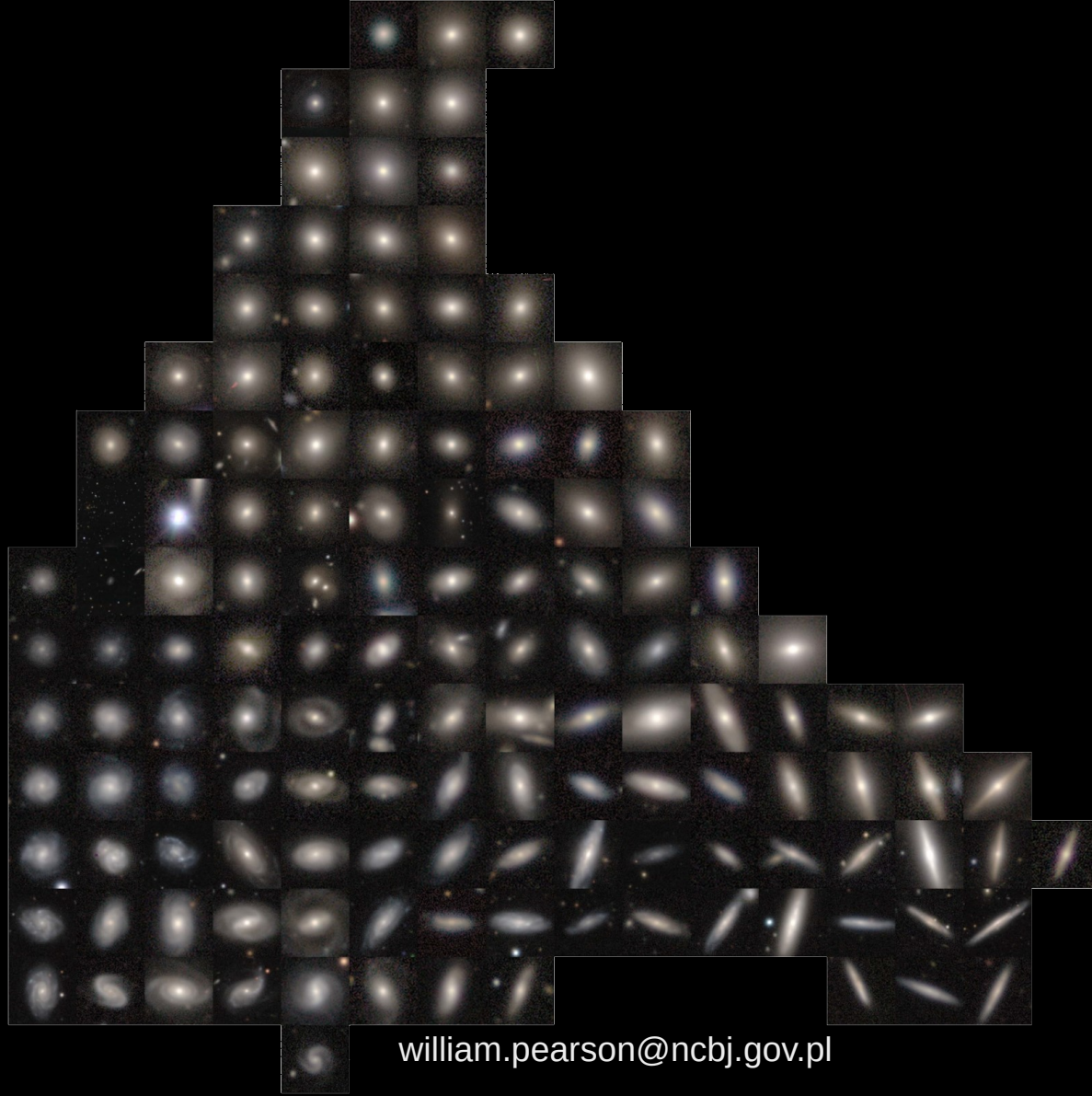
# Dimensionality Reduction

- We have a huge number of data points per object
  - Photometric measurements from Gama to Radio
    - FUV, NUV, u, g, r, i, Y, J, H, Ks, 3.4um, 4.6um, ...
  - Photometric colours
  - Redshifts
  - At least 25 morphological parameters for galaxies

# Dimensionality Reduction

- May want to extract information from known (and unknown) correlations
- Humans are TERRIBLE dealing with high dimensional data
- So we may want to reduce the number of dimensions to something more manageable





[william.pearson@ncbj.gov.pl](mailto:william.pearson@ncbj.gov.pl)

Walmsley et al. 2022

# Methods

- Autoencoder
- Self Ordering Map
- TSVD
- PCA
- tSNE
- UMAP
  - Isomap
  - SRP
  - ...

# Neural Networks

- Alex introduced neurons on Tuesday and Antonio introduced neural networks yesterday
  - $y = \max(0, \overline{xw} + b)$ 
    - Just many, many, many times
- You were shown supervised learning
  - Train the NN with labelled data
- Neural Networks can be trained in an unsupervised manner

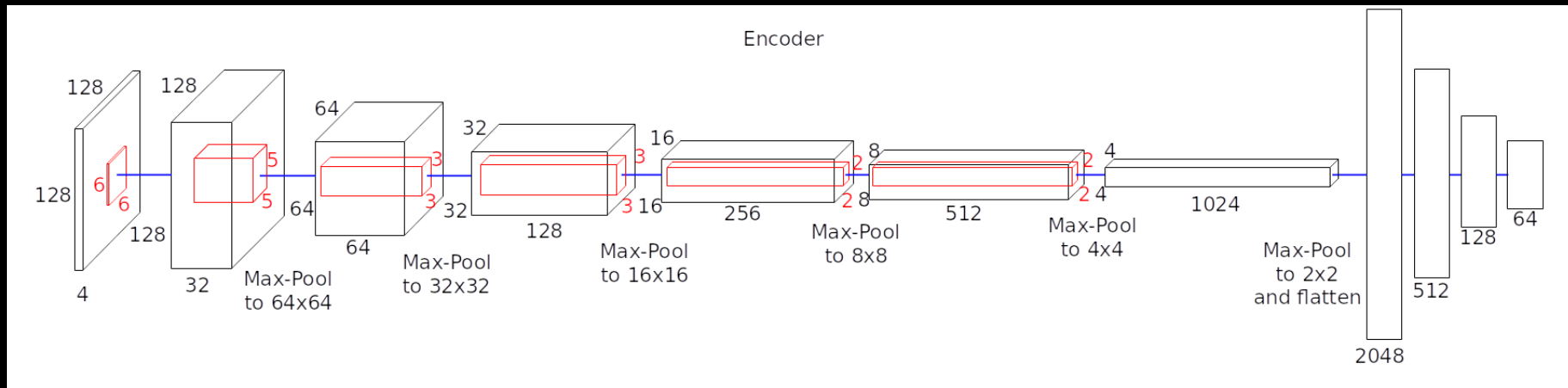


# Autoencoder

- NNs you have seen condense high dimensional data into some number of classes
  - This is basically supervised dimensionality reduction

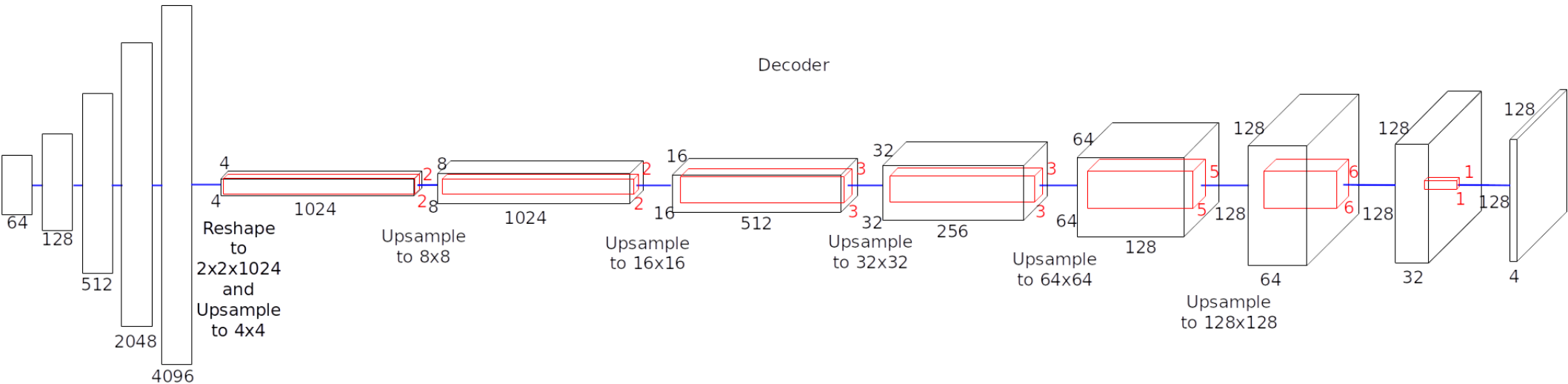
# Autoencoder

- Autoencoders try to reproduce the input data
- Use a “standard” NN to compress the data into a lower dimensional “latent space”
  - Encoder

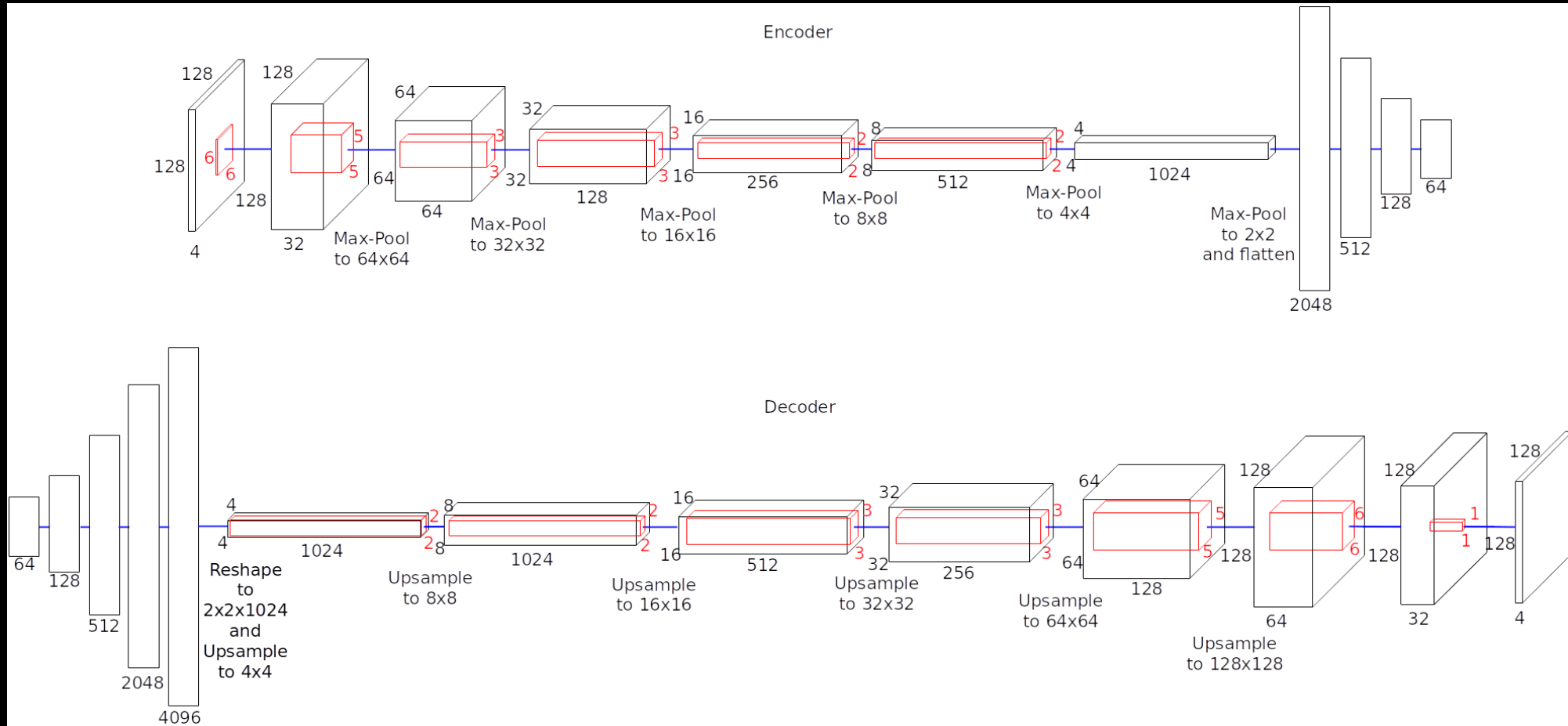


# Autoencoder

- Use a second structure to expand the latent space back into the original input
  - Decoder



# Autoencoder



# Autoencoder

- Train the encoder and decoder together
  - Do not use the latent space for determining the loss
- Loss function compares the input to the output
  - Mean Squared Error
  - Mean or Median absolute deviation



# Autoencoder

- The latent space is the dimensionally reduced parameter space
- Should contain the most useful information about the input data
  - the information that is most important for recreating the input data

# Autoencoder

- Like all NNs, overfitting can be a problem
  - especially if the autoencoder is large
- Limited data means the reconstruction of the data can be noisy
  - means the latent space may not contain the correct information
- Needs a lot of data to train
- Struggles when the source of the data changes (domain adaption)



# Self Ordering Map

- Maps high dimensional space to lower dimensional space
  - usually maps to 2D or 3D
- A neural network but trained differently to the ones seen so far

# SOM

- Say we have  $n$  objects with  $m$  parameters
  - Our “input space” is  $m$  dimensional
- Want to map into  $d$  dimensions
  - where  $d < m$
- Create a  $d$  dimensional grid that has  $j$  points (in total)
- Each of the  $j$  points has a random mapping into the input space (weights)

# SOM

- For example with 3D input space and 2D map space
  - map space  $z_1 = (0,1)$  and is initialised in the input space at  $(5,9,7)$
  - So  $\bar{z}_1$  has the weight  $w_1 = (5,9,7)$
- When training the SOM,  $w_j$  are moved in the input space, but  $z_j$  are fixed in map space
  - $z_j$  cannot move

# SOM

- For each  $x_i$  find the closest  $w_j$  (Best Matching Unit)
  - $BMU = \arg \min_j \|x_i - w_j\|$
- Update BMU's position (move it closer to  $x_i$ )
  - $w_{BMU, new} = w_{BMU} + \alpha (x_i - w_{BMU})$
  - where  $\alpha$  is the “learning rate”

# SOM

- Also update the neighbours to BMU, and their neighbours, and so on
  - $w_{j,new} = w_j + \alpha h(j, BMU)(x_i - w_j)$
  - $h(j, BMU)$  is smaller the further  $w_j$  is from  $w_{BMU}$ 
    - e.g.  $h(j, BMU)$  is 1/2 for neighbours, 1/4 for their neighbours, and so on ( $1/2^s$ )
    - or e.g.  $h(j, BMU)$  is Gaussian
- Repeat the process



# SOM

- At each iteration,  $\alpha$  is reduced based on the learning schedule
- SOMs are often run for a fixed number of iterations
- Or until some criteria is met
- Can split each iteration into batches, train on the batch then update  $\alpha$



# SOM

- Data that is similar in input space should end up clustered in map space
- New data from the input space can be placed into the map space by finding the BMU for the new data point

# Neural Networks

- Data hungry: the more training data you have, the better the network is
  - But there is diminishing returns
- Computationally heavy and require expensive hardware to run quickly

# Single Value Decomposition

- Any  $n \times m$  matrix ( $\mathbf{X}$ ) can be decomposed into
  - An  $n \times n$  unitary matrix ( $\mathbf{U}$ )
  - An  $n \times m$  diagonal matrix ( $\mathbf{\Sigma}$ ) that contains only positive, real values
  - The transpose of an  $m \times m$  complex unitary matrix ( $\mathbf{V}^T$ )
- $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$

# SVD

- The diagonal elements of  $\Sigma$  are the “single values”
- By convention, the single values are in descending order, meaning  $\Sigma$  is unique for  $\mathbf{X}$ 
  - This ordering also gives importance
- $\mathbf{V}$  contains the eigenvectors

# Truncated SVD

- We have  $n$  objects with  $m$  parameters that we want to reduce to  $d$  dimensions
- We can truncate the SVD to do this:
  - $\mathbf{X} \approx \mathbf{U}_d \mathbf{\Sigma}_d \mathbf{V}_d^T$ 
    - $\mathbf{U}_d$  is  $n \times d$
    - $\mathbf{\Sigma}_d$  is  $d \times d$
    - $\mathbf{V}_d^T$  is  $d \times m$

# TSVD

- Can use these truncated matrices to give the new, lower dimension values ( $\mathbf{Z}$ )
- $\mathbf{Z} = \mathbf{U}_d \Sigma_d$ 
  - Where  $\mathbf{Z}$  is  $n \times d$

# Principle Component Analysis

- A transform that can be applied to a matrix that identifies the direction of maximum variance
- Principle components are the eigenvectors of the covariance matrix
- Linearly projects  $m$  parameters into fewer  $d$  dimensions

# PCA

- Basically want a transformation where the new axes are ordered by variance along the axes
  - $Z = XV$
- We want the eigenvectors of the covariance matrix (**C**) of **X**

# PCA

- First, centre the data
  - $\mathbf{X}_c = \mathbf{X} - \bar{\mathbf{X}}$
- Calculate  $\mathbf{C}$ 
  - $\mathbf{C} = \mathbf{X}_c^T \mathbf{X}_c$
- Eigenvectors of  $\mathbf{C}$ 
  - $\mathbf{C}\underline{v}_i = \lambda_i \underline{v}_i$
  - $\mathbf{C}\mathbf{V} = \mathbf{\Lambda}\mathbf{V}$
  - $\mathbf{C} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$
- Using  $\lambda_i$  in descending order gives order of most variance

# PCA

- From SVD, we know
  - $\mathbf{C} = (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)^T (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)$
  - $\mathbf{C} = \mathbf{V}\mathbf{\Sigma}\mathbf{U}^T\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
  - $\mathbf{C} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^T$
- Therefore
  - $\mathbf{\Lambda} = \mathbf{\Sigma}^2$
  - $\mathbf{\Sigma} = \sqrt{\mathbf{\Lambda}}$

# PCA

- Also from SVD
  - $\mathbf{Z} = \mathbf{X}_c \mathbf{V} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T \mathbf{V}$
  - $\mathbf{Z} = \mathbf{U} \mathbf{\Sigma}$
- So...
- $\mathbf{V}$  from SVD of  $\mathbf{X}_c$  gives the coordinate transforms for PCA while  $\mathbf{\Sigma}$  gives the order
- To reduce to  $d$  dimensions, take the first  $d$  vectors from  $\mathbf{V}$

# Matrix transforms

- TSVD and PCA are matrix transformations
  - There are others, like Kernel PCA, Independent component analysis, Non-negative matrix factorisation
- Work on linear data
- Tend to approximate the data – they remove dimensions that contain less correlated information

# Manifold transformations

- Manifold – space that looks locally Euclidean (flat) but is not actually Euclidean on larger scales
- Can deal with highly non-linear data

# t-Distributed Stochastic Neighbour Embedding

- Finds the probability that two objects in high dimensional space are neighbours
- Creates a lower dimensional space (2 or 3 dimensions) where those two objects have similar probability to be neighbours

# tSNE

- The the probability that point  $j$  is a neighbour to point  $i$  in high dimensional space is given as

$$p_{j|i} = \frac{\exp\left(-\frac{|x_i - x_j|^2}{2\sigma_i^2}\right)}{\sum_{k \neq j} \exp\left(-\frac{|x_i - x_k|^2}{2\sigma_i^2}\right)}$$

with  $p_{i|i} = 0$  while the denominator ensures  $\sum_j p_{j|i} = 1$

- $\sigma_i$  is defined individually for each point so the entropy is the same for all points

# tSNE

- As  $p_{j|i}$  will not necessarily equal  $p_{i|j}$  the probabilities are made to be symmetric

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2m}$$

- tSNE now randomly places all the point  $x_{1,2,\dots,m}$  into the lower dimension space  $z_{1,2,\dots,m}$

# tSNE

- The the probability that point  $j$  is a neighbour to point  $i$  in **low** dimensional space is given as

$$q_{ij} = \frac{(1 + |z_i - z_j|^2)^{-1}}{\sum_{k \neq l} (1 + |z_k - z_l|^2)^{-1}}$$

with  $q_{i|i} = 0$

- Using a student t-distribution lets dissimilar objects be far apart in low dimensional space

# tSNE

- Now we have probabilities in high dimensional space ( $P = \{p_{ij}\}$ ) and low dimensional space ( $Q = \{q_{ij}\}$ )
- We want these two probabilities to be similar (or the same)
- Define similarity with Kullback-Leibler divergence

$$KL(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

- Move points around in low dimensional space to minimise KL using gradient descent



# tSNE

- Similar points should cluster together

BUT!

- Larger distance between clusters does not mean they are more different
- The similarity in local regions is preserved, not the global similarity

# Uniform Manifold Approximation and Projection

- Works similarly to tSNE
  - Describe the separation between points in high dimension space
  - Try to reproduce these separations in low dimension space
- But achieves this differently to tSNE

# UMAP

- There exists a manifold where all points are equally spaced apart
- Therefore, a volume centred on  $x_i$  and containing  $k$  points will have the same volume regardless of what  $i$  is
  - So we can work out distances based on the distance from  $x_i$  to the  $k$ th-neighbour
  - but each  $x_i$  has unique (local) distance measurements

# UMAP

- So instead of using (local) distances between points, assign “weights” between points ( $w_{ij}$ )
  - Similar concept to tSNE’s  $p_{j|i}$

# UMAP

- For each  $x_i$  define  $\rho_i$  and  $\sigma_i$ 
  - $\rho_i = \min\{d(x_i, x_{ij}) \mid 1 \leq j \leq k, d(x_i, x_{ij}) > 0\}$
  - $\sigma_i$  so that 
$$\sum_{j=1}^k \exp\left(\frac{-\max(0, d(x_i, x_{ij}) - \rho_i)}{\sigma_i}\right) = \log_2 k$$
- Define 
$$w_{ij} = \exp\left(\frac{-\max(0, d(x_i, x_{ij}) - \rho_i)}{\sigma_i}\right)$$

# UMAP

- The points are then randomly placed in low dimensional space
- An attractive “force” is applied between points in low dimensional space

$$\frac{-2ab\|z_i - z_j\|_2^{2(b-1)}}{1 + \|z_i - z_j\|_2^2} w_{ij}(z_i - z_j)$$

where  $a$  and  $b$  are hyper-parameters

# UMAP

- And a repulsive “force”

$$\frac{2b}{(\epsilon + \|z_i - z_j\|_2^2)(1 + a\|z_i - z_j\|_2^{2b})} (1 - w_{ij})(z_i - z_j)$$

where  $\epsilon$  is a small number to prevent dividing by 0

- These forces are then balanced with cross entropy

# UMAP

- When run, the user can choose:
  - the number of dimensions  $d$
  - the number of neighbours  $k$
  - the desired separation between close points in the low dimension space *min-dist*
  - the number of epochs to run (how long to wait for the result to converge) *n-epochs*

# UMAP

- $k$ 
  - Larger  $k$  loses smaller scale features
  - Smaller  $k$  loses larger scale features/connections
- *min-dist*
  - Larger *min-dist* makes individual points easier to see
  - Smaller *min-dist* make lower dimensional space a better representation of the high dimensional space
  - Mainly aesthetic



# UMAP

- Conceptually similar to tSNE
- But better preserves global similarity

# Applications

- Visualisation
  - human brains are terrible with more than 3 (maybe 4) dimensions
- Interpretation
  - Similar objects *should* cluster together
  - BUT the new dimensions do not necessarily mean anything
- Preparation for other methods
  - Input to neural networks
  - Input to clustering algorithms...